Oct 27, 2025

Selected Topics in Computational Quantum Physics

量子物理计算方法选讲

Shuo Yang（杨硕）

Department of Physics, Tsinghua University

Email: shuoyang@tsinghua.edu.cn

WeChat: condmat-ys

Density Matrix Renormalization Group and Matrix Product States

- origin of Density Matrix Renormalization Group method (DMRG)
- many-body entanglement
- traditional DMRG method
- Matrix Product State (MPS) and Matrix Product Operator (MPO)
- MPS algorithms
- various applications

Origin of DMRG

- Exact Diagonalization (ED) is limited by system size, how can we overcome this problem?

- the key is to truncate the Hilbert space

  challenge: select a basis that will work in general

- Numerical Renormalization Group (NRG) (1975)

  successfully applied to Kondo impurity problem

  but NRG fails for systems other than Kondo model

  K.G. Wilson, RMP 47, 773 (1975).



Ken Wilson

- how to fix it?

  Density Matrix Renormalization Group (DMRG) (1992)

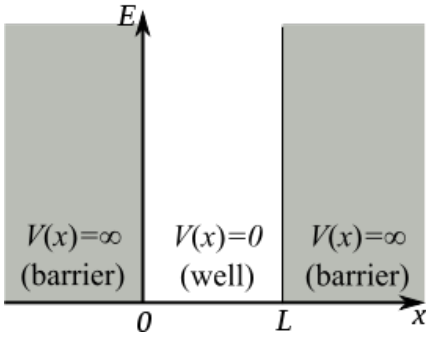
  S. R. White and R. M. Noack, PRL 68, 3487 (1992).
  S. R. White, PRL 69, 2863 (1992).
  S. R. White, PRB 48, 10345 (1993).



Steven R. White

Particle in a box problem

- we consider the particle in a box problem in 1D



$$H\psi(x) = -\frac{\hbar^2}{2m}\frac{d^2\psi(x)}{dx^2} + V(x)\psi(x) = E\psi(x)$$

- to solve the Schroedinger equation numerically, we may discretize the Hamiltonian using the standard central-difference formula for the second derivative

$$\frac{d^2\psi(x)}{dx^2} = \frac{1}{a^2}\left[\psi(x-a) - 2\psi(x) + \psi(x+a)\right]$$

- the discretized version of the above Hamiltonian is

$$H = -\sum_{l=1}^{L-1}\left(|l\rangle\langle l+1| + |l+1\rangle\langle l|\right) + 2\sum_{l=1}^{L}|l\rangle\langle l|$$

in a matrix form

e.g., $L = 5$

$$H = \begin{pmatrix} 2 & -1 & \ddots & \cdots & 0 \\ -1 & 2 & -1 & \ddots & \vdots \\ \ddots & -1 & 2 & -1 & \ddots \\ \vdots & \ddots & -1 & 2 & -1 \\ 0 & \cdots & \ddots & -1 & 2 \end{pmatrix}$$

Apply Numerical RG method

$$H = \left(\begin{array}{cc|cc|c} H & T & \ddots & \cdots & 0 \\ T^\dagger & H & T & \ddots & \vdots \\ \hline \ddots & T^\dagger & H & T & \ddots \\ \vdots & \ddots & T^\dagger & H & \ddots \\ 0 & \cdots & \ddots & \ddots & \ddots \end{array}\right) = \left(\begin{array}{cc|c} H' & T' & 0 \\ T'^\dagger & H' & \ddots \\ \hline 0 & \ddots & \ddots \end{array}\right) \rightarrow \left(\begin{array}{cc|c} H'' & T'' & 0 \\ T''^\dagger & H'' & \ddots \\ \hline 0 & \ddots & \ddots \end{array}\right)$$

initially

$H = (2), T = (-1)$

- procedure:

1. combine two blocks, $H' = \begin{pmatrix} H & T \\ T^\dagger & H \end{pmatrix}, T' = \begin{pmatrix} 0 & 0 \\ T & 0 \end{pmatrix}$
2. diagonalize H', getting eigenvectors V_l, $H'V_n = E_nV_n$
3. build matrix O using low-energy eigenvectors $O = \begin{pmatrix} \vdots & \vdots & & \vdots \\ V_1 & V_2 & \cdots & V_m \\ \vdots & \vdots & & \vdots \end{pmatrix}$
4. change basis and truncate, $H'' = O^\dagger H'O, T'' = O^\dagger T'O,$
5. replace H and T by H'' and T'' and iterate

H'' = $O^\dagger$ H' O

T'' = $O^\dagger$ T' O

```
Hp = np.zeros((2,2))
Hp[0,0] = 2.0
Hp[1,1] = 2.0
Hp[0,1] = -1.0
Hp[1,0] = -1.0

Tp = np.zeros((2,2))
Tp[1,0] = -1.0

for i in range(Iter):
    S,V = LA.eig(Hp)
    order = np.argsort(S)
    S = S[order]
    V = V[:,order]
    Dc = min(len(S),M)
    S = S[:Dc]
    V = V[:,:Dc]
    # print(S,Dc)

    H = np.dot(np.dot(np.transpose(np.conj(V)),Hp),V)
    T = np.dot(np.dot(np.transpose(np.conj(V)),Tp),V)

    Ns = np.shape(H)[0]

    Hp = np.zeros((Ns*2,Ns*2))
    Hp[:Ns,:Ns] = H
    Hp[Ns:,Ns:] = H
    Hp[:Ns,Ns:] = T
    Hp[Ns:,:Ns] = np.transpose(np.conj(T))

    Tp = np.zeros((Ns*2,Ns*2))
    Tp[Ns:,:Ns] = T

print('Standard',S[:4])
```

initially

$$H' = \begin{pmatrix} H & T \\ T^\dagger & H \end{pmatrix} = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$$

$$T' = \begin{pmatrix} 0 & 0 \\ T & 0 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ -1 & 0 \end{pmatrix}$$

diagonalize H', getting eigenvectors V_l

$$H'V_n = E_nV_n$$

build matrix O using low-energy eigenvectors

$$O = \begin{pmatrix} \vdots & \vdots & & \vdots \\ V_1 & V_2 & \cdots & V_m \\ \vdots & \vdots & & \vdots \end{pmatrix}$$

$H'' = O^\dagger H' O$

$T'' = O^\dagger T' O$

replace H and T by H'' and T'' and iterate

$$H' = \begin{pmatrix} H & T \\ T^\dagger & H \end{pmatrix}, T' = \begin{pmatrix} 0 & 0 \\ T & 0 \end{pmatrix}$$

Performance of NRG method

TABLE I. Lowest energies after ten blocking transformations for the noninteracting single particle on a 1D chain with fixed boundary conditions.

	Exact	Standard (NRG)
E_0	2.3508×10^{-6}	1.9207×10^{-2}
E_1	9.4032×10^{-6}	1.9209×10^{-2}
E_2	2.1157×10^{-5}	1.9214×10^{-2}
E_3	3.7613×10^{-5}	1.9217×10^{-2}

- why NRG fails?
- particle in a box eigenstates are 0 at the edges
- any state formed by low-lying states has a kink in the middle
- to remove kink, need to keep almost all states

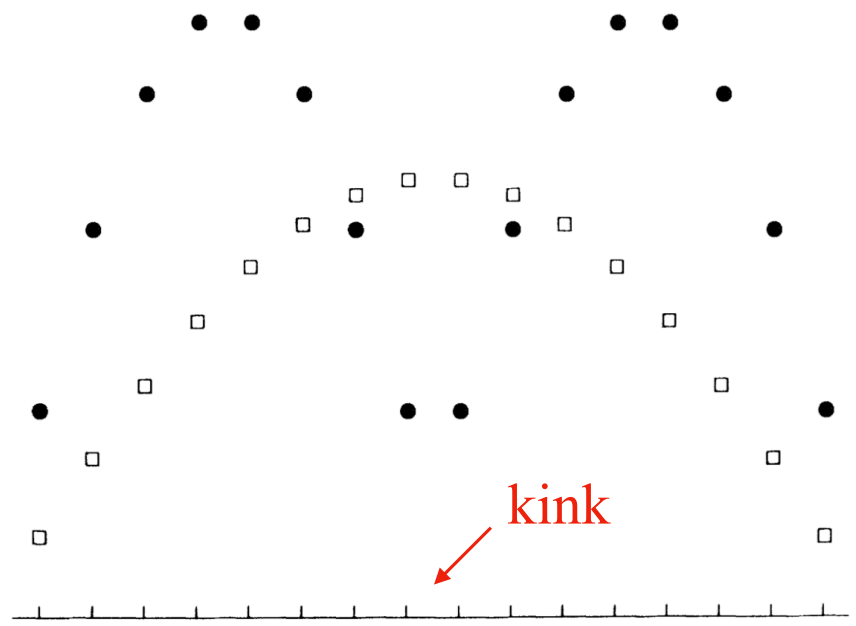


FIG. 1. Lowest eigenstates of two 8-site blocks (solid circles) and a 16-site block (open squares) for the one-dimensional tight-binding model with fixed boundary conditions.

How to fix it?

- combine states from different boundary conditions

  states must be orthogonalized

- fixed-free combination

	free	fixed
free	$H_{11}^{'[n+1]} = \begin{pmatrix} H_{12}^{[n]} & T^{[n]} \\ T^{\dagger[n]} & H_{21}^{[n]} \end{pmatrix}, H_{11}^{'[1]} = \left(\begin{array}{cc\|cc} 1 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ \hline 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 1 \end{array}\right)$	$H_{12}^{'[n+1]} = \begin{pmatrix} H_{12}^{[n]} & T^{[n]} \\ T^{\dagger[n]} & H_{22}^{[n]} \end{pmatrix}, H_{12}^{'[1]} = \left(\begin{array}{cc\|cc} 1 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ \hline 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{array}\right)$
fixed	$H_{21}^{'[n+1]} = \begin{pmatrix} H_{22}^{[n]} & T^{[n]} \\ T^{\dagger[n]} & H_{21}^{[n]} \end{pmatrix}, H_{21}^{'[1]} = \left(\begin{array}{cc\|cc} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ \hline 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 1 \end{array}\right)$	$H_{22}^{'[n+1]} = \begin{pmatrix} H_{22}^{[n]} & T^{[n]} \\ T^{\dagger[n]} & H_{22}^{[n]} \end{pmatrix}, H_{22}^{'[1]} = \left(\begin{array}{cc\|cc} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ \hline 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{array}\right)$

$$T^{'[n+1]} = \begin{pmatrix} 0 & 0 \\ T^{[n]} & 0 \end{pmatrix}, T^{'[1]} = \left(\begin{array}{cc\|cc} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 \end{array}\right)$$

S. R. White and R. M. Noack, PRL 68, 3487 (1992).

Fixed-free method

- use the lowest $m/4$ states from each of H_{11}, H_{12}, H_{21}, H_{22}

$$\widetilde{O} = \left(\begin{array}{ccc|ccc|ccc|ccc} \vdots & & \vdots & \vdots & & \vdots & \vdots & & \vdots & \vdots & & \vdots \\ V_1^{11} & \cdots & V_{m/4}^{11} & V_1^{12} & \cdots & V_{m/4}^{12} & V_1^{21} & \cdots & V_{m/4}^{21} & V_1^{22} & \cdots & V_{m/4}^{22} \\ \vdots & & \vdots & \vdots & & \vdots & \vdots & & \vdots & \vdots & & \vdots \end{array} \right)$$

$$O = \text{Gram-Schmidt}\left(\widetilde{O}\right)$$

- change basis to get new H_{11}, H_{12}, H_{21}, H_{22} and T

$$H_{11}^{[n]} = O^\dagger H_{11}'^{[n]} O, \quad H_{12}^{[n]} = O^\dagger H_{12}'^{[n]} O, \quad H_{21}^{[n]} = O^\dagger H_{21}'^{[n]} O, \quad H_{22}^{[n]} = O^\dagger H_{22}'^{[n]} O, \quad T^{[n]} = O^\dagger T'^{[n]} O$$

- results

TABLE I. Lowest energies after ten blocking transformations for the noninteracting single particle on a 1D chain with fixed boundary conditions.

	Exact	Standard	Fixed free
E_0	2.3508×10^{-6}	1.9207×10^{-2}	2.3508×10^{-6}
E_1	9.4032×10^{-6}	1.9209×10^{-2}	9.4032×10^{-6}
E_2	2.1157×10^{-5}	1.9214×10^{-2}	2.1157×10^{-5}
E_3	3.7613×10^{-5}	1.9217×10^{-2}	3.7613×10^{-5}

- connections to surrounding blocks are important!

  such method may not work for interacting systems, how to improve this?

```
H_11 = np.asarray([[1,-1],[-1,1]])
H_12 = np.asarray([[1,-1],[-1,2]])
H_21 = np.asarray([[2,-1],[-1,1]])
H_22 = np.asarray([[2,-1],[-1,2]])
T = np.asarray([[0,0],[-1,0]])

for i in range(Iter-1):
    Ns = np.shape(H_11)[0]
    Z = np.zeros((Ns,Ns))

    Hp_11 = np.vstack([np.hstack([H_12,T]),np.hstack([np.transpose(np.conj(T)),H_21])])
    Hp_12 = np.vstack([np.hstack([H_12,T]),np.hstack([np.transpose(np.conj(T)),H_22])])
    Hp_21 = np.vstack([np.hstack([H_22,T]),np.hstack([np.transpose(np.conj(T)),H_21])])
    Hp_22 = np.vstack([np.hstack([H_22,T]),np.hstack([np.transpose(np.conj(T)),H_22])])
    Tp = np.vstack([np.hstack([Z,Z]),np.hstack([T,Z])])

    S_11,V_11 = LA.eig(Hp_11)
    order = np.argsort(S_11)
    S_11 = S_11[order]
    V_11 = V_11[:,order]
    Dc = min(len(S_11),M//4)
    S_11 = S_11[:Dc]
    V_11 = V_11[:,:Dc]
    # print(S_11,Dc)

    S_12,V_12 = LA.eig(Hp_12)
    order = np.argsort(S_12)
    S_12 = S_12[order]
    V_12 = V_12[:,order]
    Dc = min(len(S_12),M//4)
    S_12 = S_12[:Dc]
    V_12 = V_12[:,:Dc]
    # print(S_12,Dc)
```

```
    S_21,V_21 = LA.eig(Hp_21)
    order = np.argsort(S_21)
    S_21 = S_21[order]
    V_21 = V_21[:,order]
    Dc = min(len(S_21),M//4)
    S_21 = S_21[:Dc]
    V_21 = V_21[:,:Dc]
    # print(S_21,Dc)

    S_22,V_22 = LA.eig(Hp_22)
    order = np.argsort(S_22)
    S_22 = S_22[order]
    V_22 = V_22[:,order]
    Dc = min(len(S_22),M//4)
    S_22 = S_22[:Dc]
    V_22 = V_22[:,:Dc]
    # print(S_22,Dc)
```

$$\tilde{O} = \left(\begin{array}{ccc|ccc|ccc|ccc} \vdots & & \vdots & \vdots & & \vdots & \vdots & & \vdots & \vdots & & \vdots \\ V_1^{11} & \cdots & V_{m/4}^{11} & V_1^{12} & \cdots & V_{m/4}^{12} & V_1^{21} & \cdots & V_{m/4}^{21} & V_1^{22} & \cdots & V_{m/4}^{22} \\ \vdots & & \vdots & \vdots & & \vdots & \vdots & & \vdots & \vdots & & \vdots \end{array} \right)$$

$$O = \text{Gram-Schmidt}\left(\tilde{O}\right)$$

```
    V = np.hstack([V_11,V_12,V_21,V_22])
    Q,R = linalg.qr(V,mode='economic')

    H_11 = np.dot(np.dot(np.transpose(np.conj(Q)),Hp_11),Q)
    H_12 = np.dot(np.dot(np.transpose(np.conj(Q)),Hp_12),Q)
    H_21 = np.dot(np.dot(np.transpose(np.conj(Q)),Hp_21),Q)
    H_22 = np.dot(np.dot(np.transpose(np.conj(Q)),Hp_22),Q)
    T = np.dot(np.dot(np.transpose(np.conj(Q)),Tp),Q)

S_22,V_22 = LA.eig(Hp_22)
order = np.argsort(S_22)
S_22 = S_22[order]
V_22 = V_22[:,order]
print('Fixed free',S_22[:4])
```

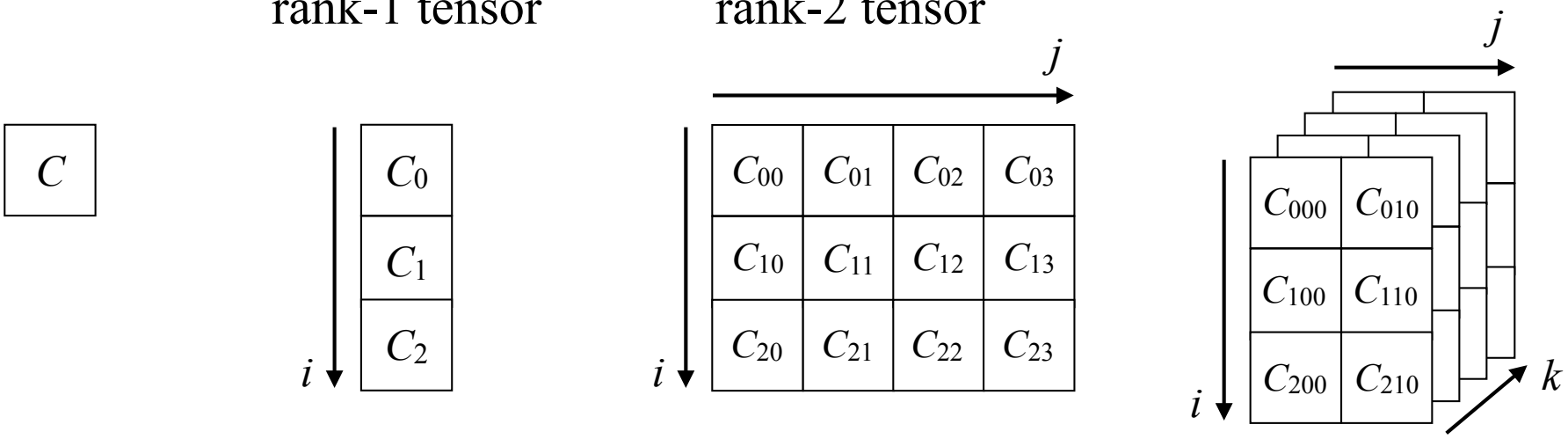
Tensor and its graphical notation

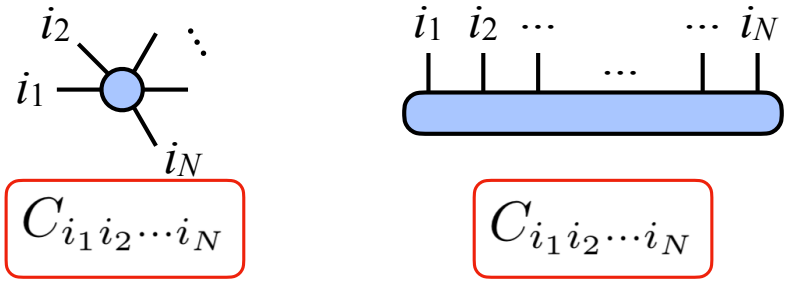
- Penrose graphical notation



constant | vector | matrix | rank-3 tensor

rank-1 tensor | rank-2 tensor





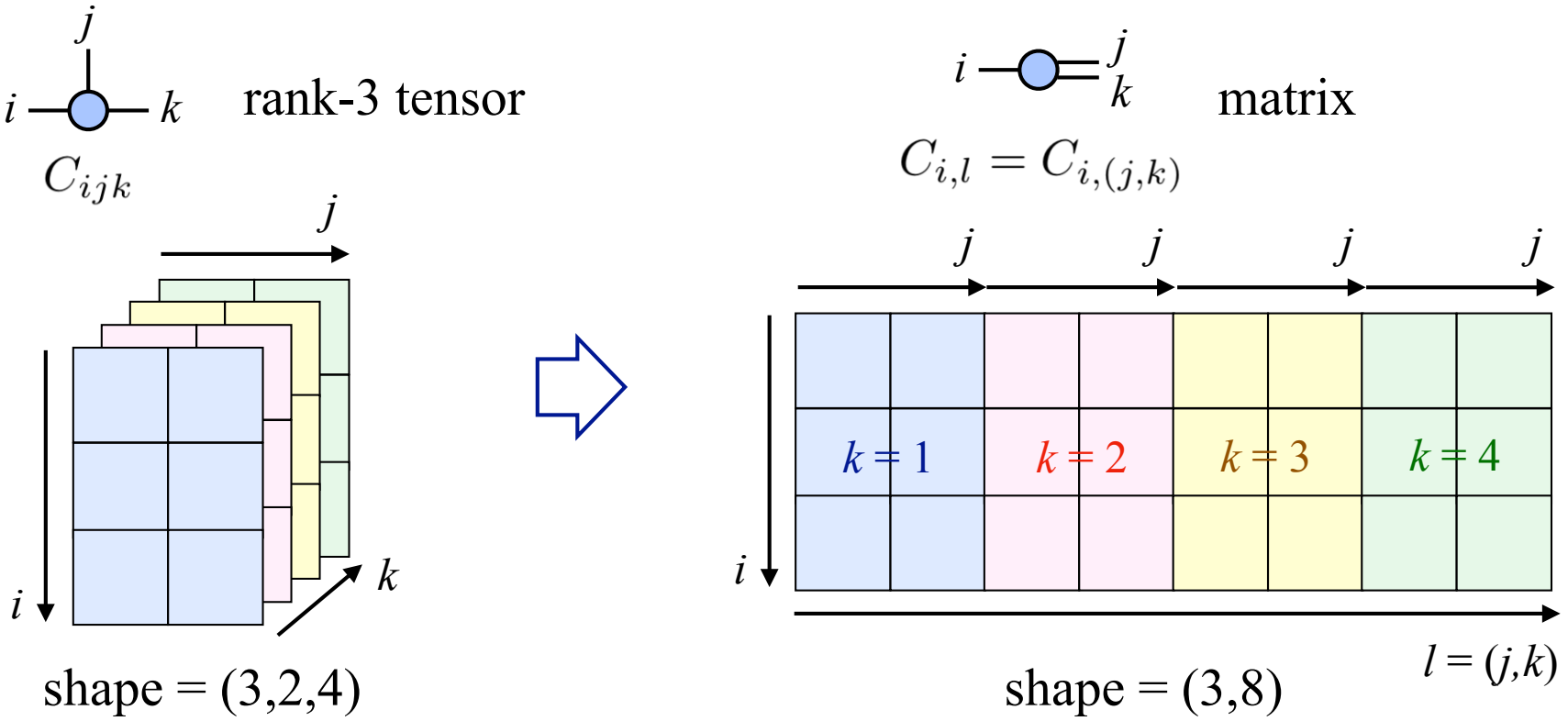
rank-N tensor | rank-N tensor

- an N-spin quantum state

$$|\psi\rangle = \sum_{i_1 i_2 \cdots i_N} C_{i_1 i_2 \cdots i_N} |i_1 i_2 \cdots i_N\rangle$$

is a rank-N tensor

Reshape a tensor



- python code

```
In [1]: import numpy as np

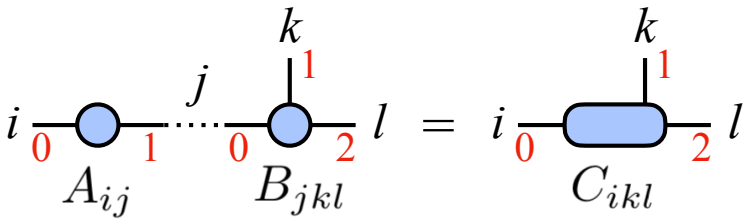
In [2]: C = np.random.rand(3,2,4)

In [3]: C = np.reshape(C,[3,8])

In [4]: np.shape(C)
Out[4]: (3, 8)
```

Tensor contraction

- connecting two index lines implies a contraction, or summation over the same indices



$$\sum_j A_{ij} B_{jkl} = C_{ikl}$$

- python code

```
In [2]: import numpy as np

In [3]: A = np.random.rand(3,2)

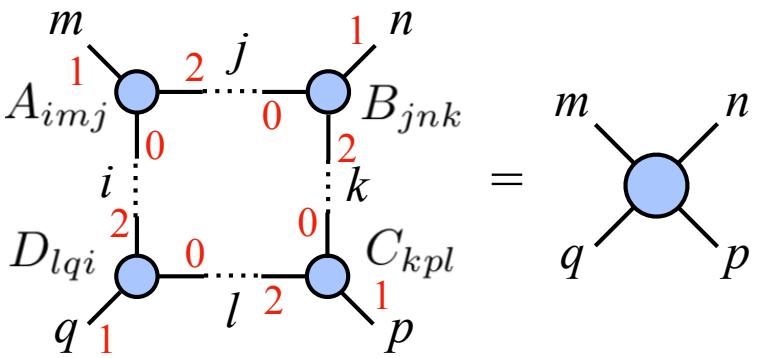
In [4]: B = np.random.rand(2,4,3)

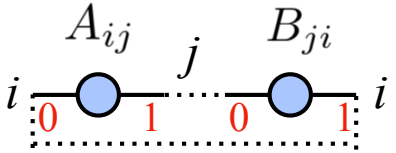
In [5]: C = np.tensordot(A,B,(1,0))

In [6]: np.shape(C)
Out[6]: (3, 4, 3)
```

→ contract the 1st index of A and the 0th index of B

- other examples

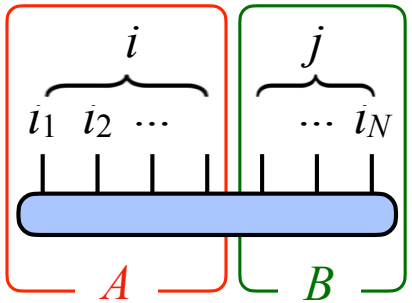


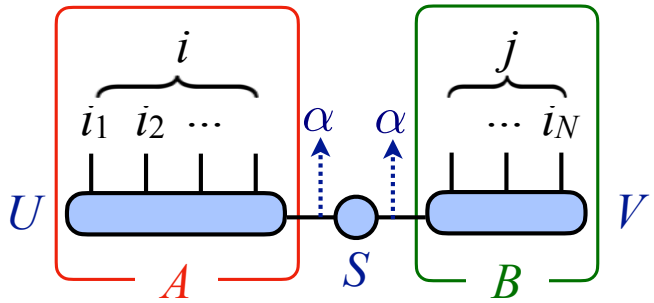


$$\sum_{ij} A_{ij} B_{ji} = \mathrm{Tr}\,(AB)$$

Schimidt decomposition

- bipartition of a quantum system in blocks A and B with orthogonal bases





$$|\psi\rangle = \sum_{i_1,i_2,\cdots,i_N} C_{i_1,i_2,\cdots,i_N} |i_1, i_2, \cdots, i_N\rangle$$

$$= \sum_{ij} C_{ij} |i\rangle_A |j\rangle_B$$

$|i\rangle_A$ and $|j\rangle_B$ form orthogonal bases

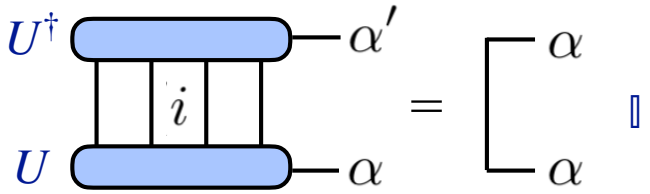
$$|\psi\rangle = \sum_{\alpha} s_\alpha |u^\alpha\rangle_A |v^\alpha\rangle_B$$

$$= \sum_{i,j} \left[\sum_{\alpha} s_\alpha u_i^\alpha v_j^\alpha \right] |i\rangle_A |j\rangle_B$$

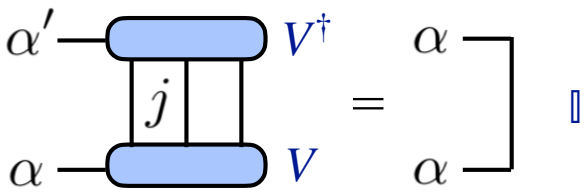
$|u^\alpha\rangle_A$ and $|v^\alpha\rangle_B$ form orthogonal bases

- graphical notation of the orthogonal condition

$$\sum_i u_i^{\alpha *} u_i^{\alpha'} = \delta_{\alpha\alpha'}, \qquad U^\dagger U = \mathbb{I}$$



$$\sum_i v_j^{\alpha *} v_j^{\alpha'} = \delta_{\alpha\alpha'}, \qquad V V^\dagger = \mathbb{I}$$



Singular Value Decomposition (SVD)

- Schimidt decomposition can be obtained using Singular Value Decomposition
- for an arbitrary matrix C of dimension $(m \times n)$ there exists a decomposition

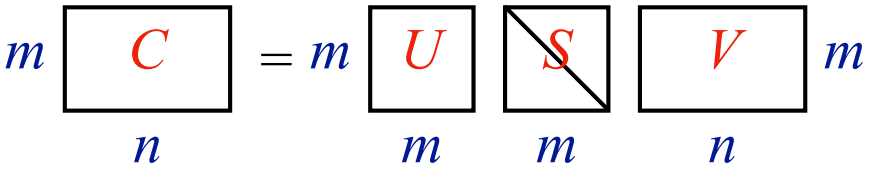
$$C = U\,S\,V$$

where S is diagonal, and U and V are unitary matrices

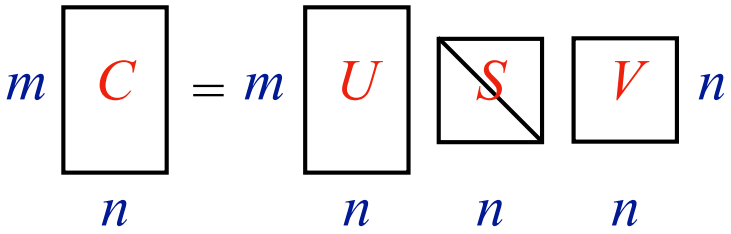
$$S = \mathrm{diag}\,(s_1, s_2, \cdots, s_r)\,, s_1 \geq s_2 \geq \cdots \geq s_r \geq 0, r = \min\,(m, n)$$

$$UU^\dagger = U^\dagger U = \mathbb{I}, VV^\dagger = V^\dagger V = \mathbb{I} \qquad \text{when } m = n$$

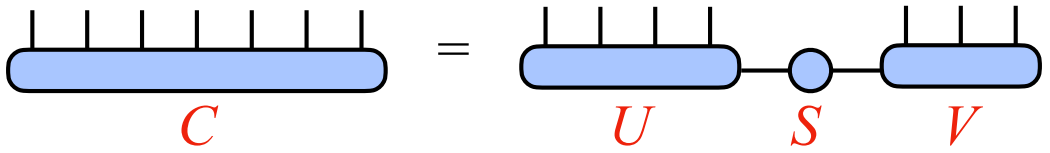
- when $m < n$



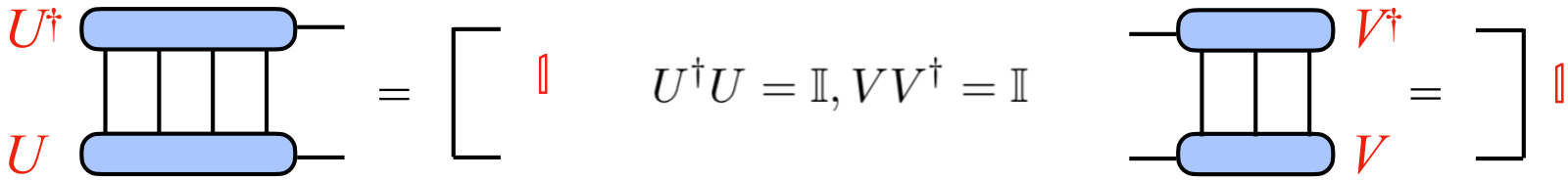
- when $m > n$



- SVD is equivalent to Schimidt decomposition



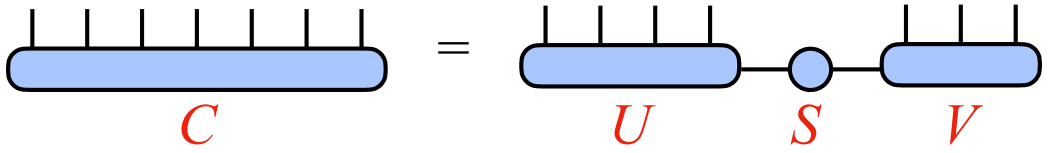
with



$$U^\dagger U = \mathbb{I}, VV^\dagger = \mathbb{I}$$

Reduced density matrix

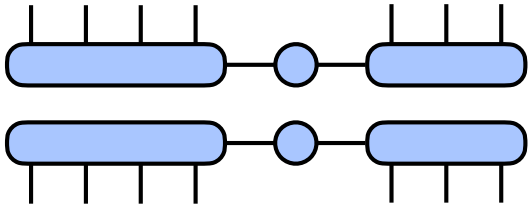
- Schmidt decomposition



$$|\psi\rangle = \sum_{ij} C_{ij} |i\rangle_A |j\rangle_B = \sum_{\alpha} s_\alpha |u^\alpha\rangle_A |v^\alpha\rangle_B$$

- density matrix of the whole system

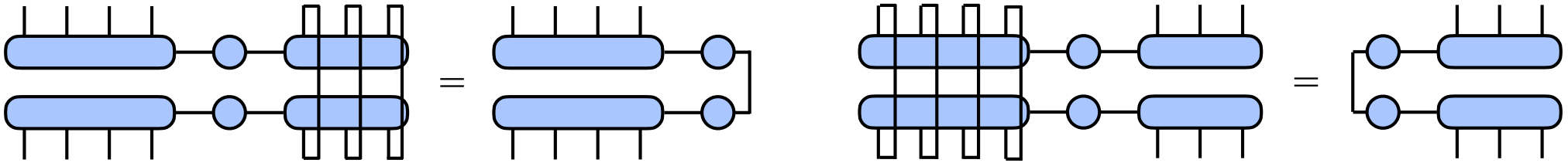
$$\rho = |\psi\rangle\langle\psi|$$



- reduced density matrix

$$\rho_A = \mathrm{Tr}_B \rho = \sum_{\alpha} s_\alpha^2 |u^\alpha\rangle_A \, {}_A\langle u^\alpha|$$

$$\rho_B = \mathrm{Tr}_A \rho = \sum_{\alpha} s_\alpha^2 |v^\alpha\rangle_B \, {}_B\langle v^\alpha|$$



Entanglement entropy

- Entanglement Entropy (EE) is a measure of the amount of entanglement

$$S = -\mathrm{Tr}[\rho_A \ln \rho_A] = -\mathrm{Tr}[\rho_B \ln \rho_B] = -\sum_\alpha s_\alpha^2 \ln s_\alpha^2$$

- example 1: product state

$$|\psi\rangle = \frac{1}{2}\left(|\uparrow\rangle_A + |\downarrow\rangle_A\right)\left(|\uparrow\rangle_B + |\downarrow\rangle_B\right)$$

$$\rho_A = {}_B\langle\uparrow|\left(|\psi\rangle\langle\psi|\right)|\uparrow\rangle_B + {}_B\langle\downarrow|\left(|\psi\rangle\langle\psi|\right)|\downarrow\rangle_B = \frac{1}{2}\left(|\uparrow\rangle_A + |\downarrow\rangle_A\right)\left({}_A\langle\uparrow| + {}_A\langle\downarrow|\right)$$

$$\rho_A = \frac{1}{2}\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} = \left[\frac{1}{\sqrt{2}}\begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}\right]\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}\left[\frac{1}{\sqrt{2}}\begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}\right]^\dagger$$

$$S = -1 \times \ln 1 - 0 \times \ln 0 = 0$$ one non-zero Schmidt value

- example 2: maximally entangled state

$$|\psi\rangle = \frac{1}{\sqrt{2}}\left(|\uparrow\rangle_A|\downarrow\rangle_B + |\downarrow\rangle_A|\uparrow\rangle_B\right)$$

$$\rho_A = {}_B\langle\uparrow|\left(|\psi\rangle\langle\psi|\right)|\uparrow\rangle_B + {}_B\langle\downarrow|\left(|\psi\rangle\langle\psi|\right)|\downarrow\rangle_B = \frac{1}{2}|\downarrow\rangle_A\,{}_A\langle\downarrow| + \frac{1}{2}|\uparrow\rangle_A\,{}_A\langle\uparrow| = \frac{1}{2}\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

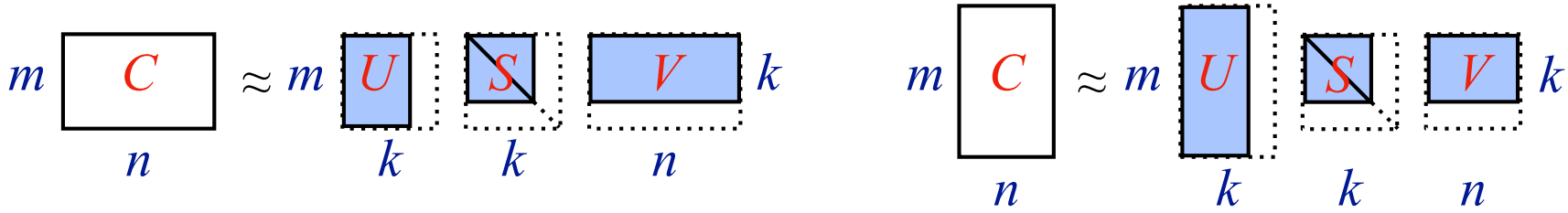
$$S = -\frac{1}{2}\ln\frac{1}{2} - \frac{1}{2}\ln\frac{1}{2} = \ln 2$$ two non-zero Schmidt values

Truncation in SVD

- we want to minimize ε over all s_α, u_i^α, and v_j^α

$$\varepsilon = \left||\psi\rangle - |\bar{\psi}\rangle\right|^2 = \sum_{ij}\left(C_{ij} - \sum_\alpha s_\alpha u_i^\alpha v_j^\alpha\right)^2 = |C - USV|^2$$

- the solution is produced by making a truncation in SVD



$$U^\dagger U = \mathbb{I}, VV^\dagger = \mathbb{I}$$

- different levels of truncation, e.g., an image is stored as a 426*640 matrix

image compression with SVD



$k = 426$, no truncation



$k = 40$

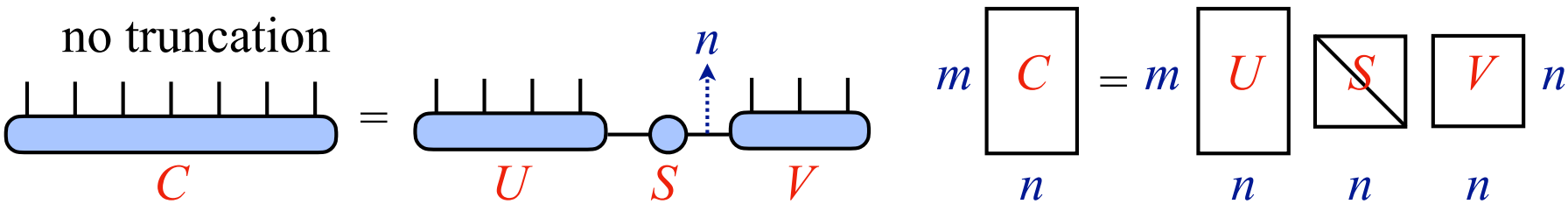


$k = 20$

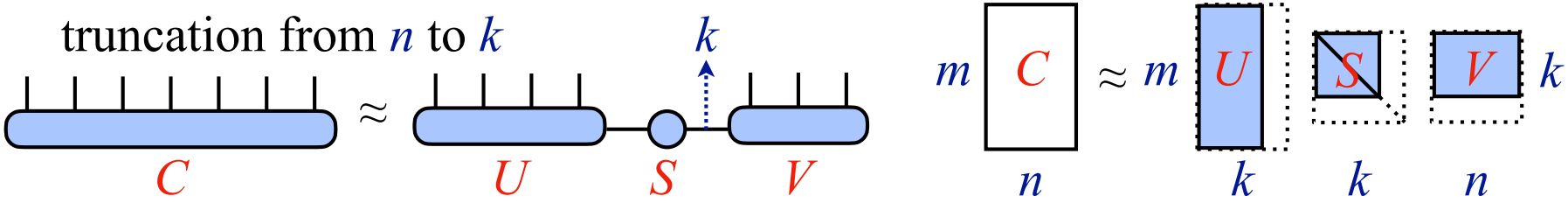
Truncation in eigenvalue decomposition (EVD)

- we have shown how to minimize ε using SVD

no truncation

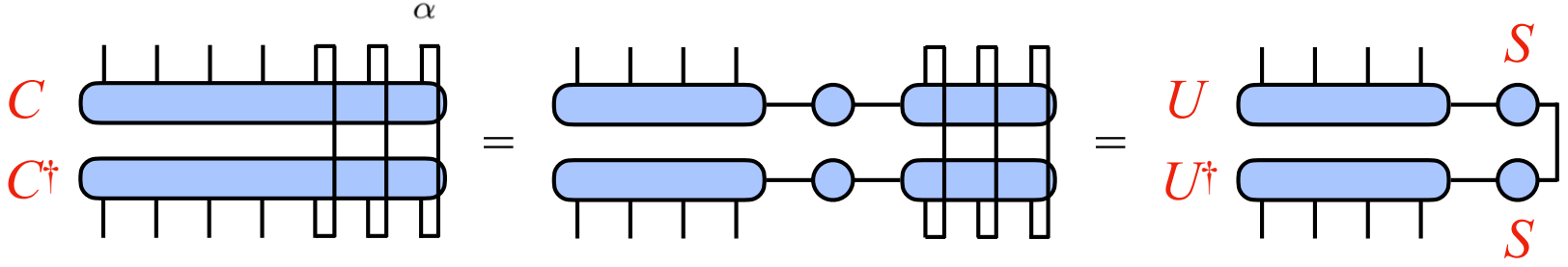


truncation from n to k

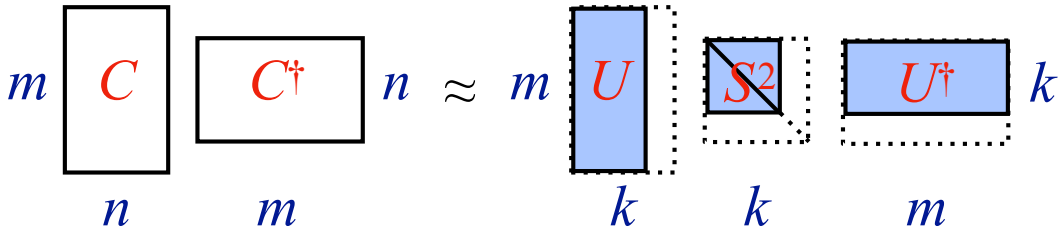


- equivalently, we can minimize ε using eigenvalue decomposition (EVD)

$$\rho_A = \mathrm{Tr}_B \rho = \sum_\alpha s_\alpha^2 \, |u^\alpha\rangle_A \, {}_A\langle u^\alpha| = CC^\dagger = USVV^\dagger SU^\dagger = US^2U^\dagger$$



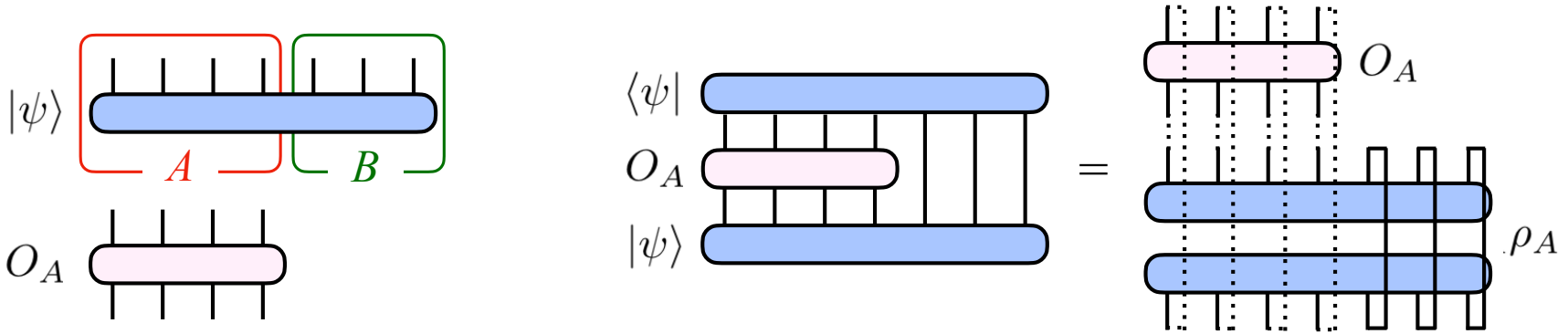
truncation from n to k



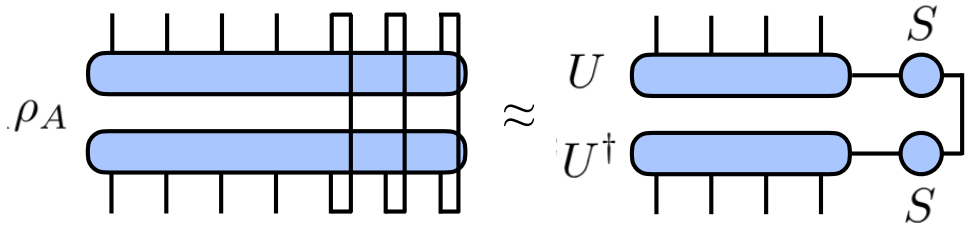
Density matrix projection for interacting systems

- given $|\psi\rangle$ and an operator O_A acting on the A part, the expectation value is

$$\langle O_A \rangle = \langle \psi | O_A | \psi \rangle = \mathrm{Tr}\,(O_A \rho_A)$$

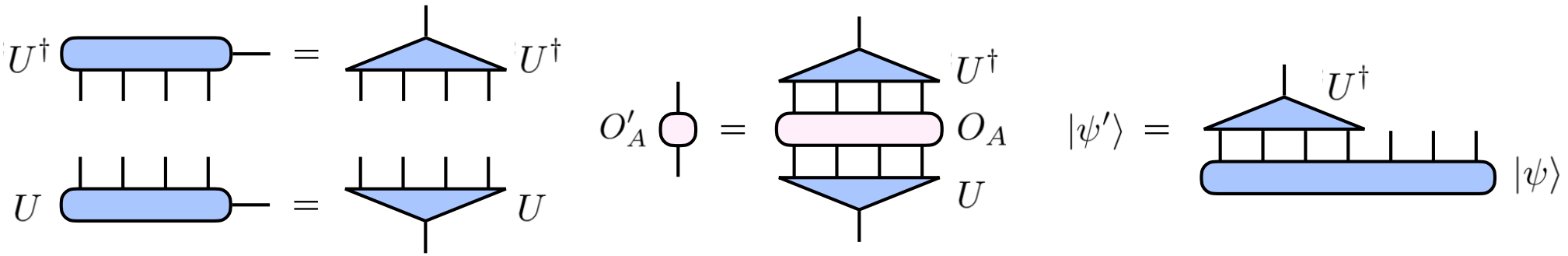


- the reduced density matrix ρ_A tells us which states are the most important

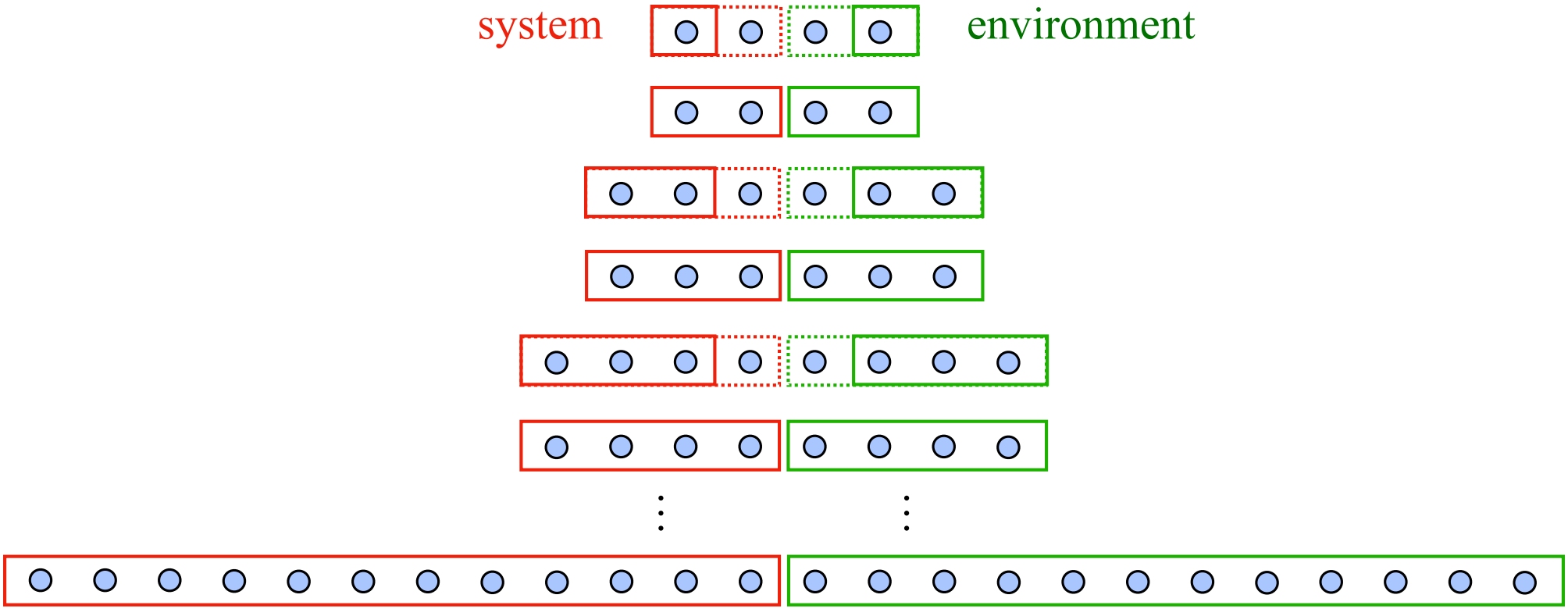


$$\rho_A \approx U S^2 U^\dagger$$

- then we use $U^\dagger$ and U to truncate the operator O_A , i.e., $O'_A = U^\dagger O_A U$

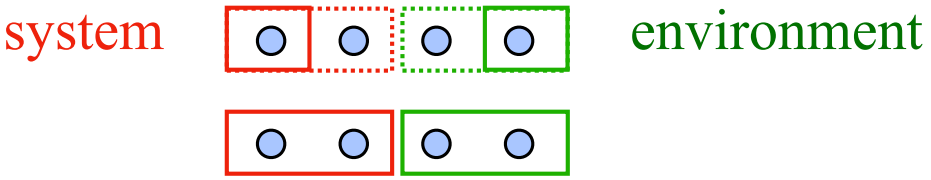


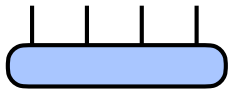
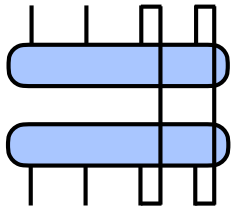
Traditional infinite DMRG algorithm

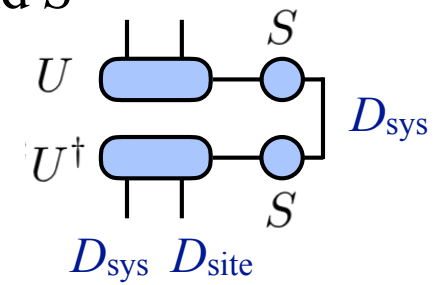


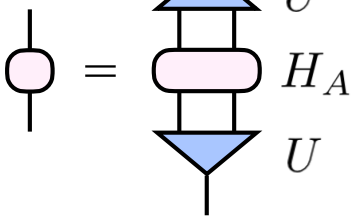
- grow system size by one site a time
- system and environment have mirror symmetry
- each time make a truncation of Hilbert space, $D_{\text{system}} \times D_{\text{site}} \rightarrow D_{\text{system}}$

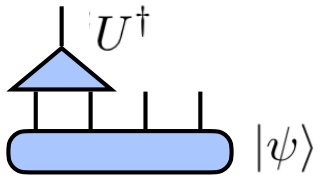
Procedure to grow one system site



- form the Hamiltonian matrix for the superblock
  (system + site + site + environment)
- diagonalize the superblock Hamiltonian to find the ground state $|\psi\rangle$ 
- form the reduced density matrix of (system + site) ρ_A 
- diagonalize ρ_A to find U and S, and make a truncation to U and S
- form the new system block using U

$\rho_A \approx$ 

H'_A 

$|\psi'\rangle =$ 

Sample code of traditional DMRG

- can be downloaded from
  https://github.com/simple-dmrg/simple-dmrg

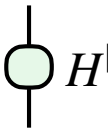
- including:
  infinite system algorithm
  finite system algorithm
  algorithm with conserved quantum numbers
  algorithm with eigenstate prediction

- example: infinite system algorithm

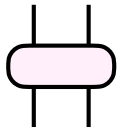
```
160   def infinite_system_algorithm(L, m):
161       block = initial_block
162       # Repeatedly enlarge the system by performing a single DMRG step, using a
163       # reflection of the current block as the environment.
164       while 2 * block.length < L:
165           print("L =", block.length * 2 + 2)
166           block, energy = single_dmrg_step(block, block, m=m)
167           print("E/L =", energy / (block.length * 2))
```

Define Hamiltonian

$$H = \sum_j \left[J \left(S_j^+ S_{j+1}^- + S_j^- S_{j+1}^+ \right) /2 + J_z S_j^z S_{j+1}^z \right]$$



$H^{[1]} \qquad H^{[1]} = 0$



$H^{[2]}$

$$H^{[2]} = J \left(S_1^+ S_2^- + S_1^- S_2^+ \right) /2 + J_z S_1^z S_2^z$$

```
# Model-specific code for the Heisenberg XXZ chain
model_d = 2  # single-site basis size

Sz1 = np.array([[0.5, 0], [0, -0.5]], dtype='d')  # single-site S^z
Sp1 = np.array([[0, 1], [0, 0]], dtype='d')  # single-site S^+

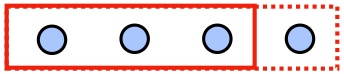
H1 = np.array([[0, 0], [0, 0]], dtype='d')  # single-site portion of H is zero

def H2(Sz1, Sp1, Sz2, Sp2):  # two-site part of H
    """Given the operators S^z and S^+ on two sites in different Hilbert spaces
    (e.g. two blocks), returns a Kronecker product representing the
    corresponding two-site term in the Hamiltonian that joins the two sites.
    """
    J = Jz = 1.
    return (
        (J / 2) * (kron(Sp1, Sp2.conjugate().transpose()) + kron(Sp1.conjugate().transpose(), Sp2)) +
        Jz * kron(Sz1, Sz2)
    )

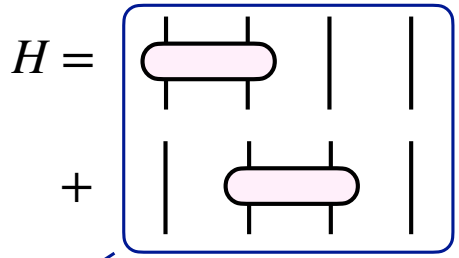
# conn refers to the connection operator, that is, the operator on the edge of
# the block, on the interior of the chain.  We need to be able to represent S^z
# and S^+ on that site in the current basis in order to grow the chain.
initial_block = Block(length=1, basis_size=model_d, operator_dict={
    "H": H1,
    "conn_Sz": Sz1,
    "conn_Sp": Sp1,
})
```

Enlarge block

- aim



- the total Hamiltonian of the enlarged block is

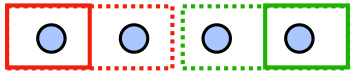



```
64  def enlarge_block(block):
65      """This function enlarges the provided Block by a single site, returning an
66      EnlargedBlock.
67      """
68      mblock = block.basis_size
69      o = block.operator_dict
70
71      # Create the new operators for the enlarged block.  Our basis becomes a
72      # Kronecker product of the Block basis and the single-site basis.  NOTE:
73      # `kron` uses the tensor product convention making blocks of the second
74      # array scaled by the first.  As such, we adopt this convention for
75      # Kronecker products throughout the code.
76      enlarged_operator_dict = {
77          "H": kron(o["H"], identity(model_d)) + kron(identity(mblock), H1) + H2(o["conn_Sz"], o["conn_Sp"], Sz1, Sp1),
78          "conn_Sz": kron(identity(mblock), Sz1),
79          "conn_Sp": kron(identity(mblock), Sp1),
80      }
81
82      return EnlargedBlock(length=(block.length + 1),
83                           basis_size=(block.basis_size * model_d),
84                           operator_dict=enlarged_operator_dict)
```

DMRG step (1)

```
92  def single_dmrg_step(sys, env, m):
93      """Performs a single DMRG step using `sys` as the system and `env` as the
94      environment, keeping a maximum of `m` states in the new basis.
95      """
96      assert is_valid_block(sys)
97      assert is_valid_block(env)
98
99      # Enlarge each block by a single site.
100     sys_enl = enlarge_block(sys)
101     if sys is env:  # no need to recalculate a second time
102         env_enl = sys_enl
103     else:
104         env_enl = enlarge_block(env)
105
106     assert is_valid_enlarged_block(sys_enl)
107     assert is_valid_enlarged_block(env_enl)
108
109     # Construct the full superblock Hamiltonian.
110     m_sys_enl = sys_enl.basis_size
111     m_env_enl = env_enl.basis_size
112     sys_enl_op = sys_enl.operator_dict
113     env_enl_op = env_enl.operator_dict
114     superblock_hamiltonian = kron(sys_enl_op["H"], identity(m_env_enl)) + kron(identity(m_sys_enl), env_enl_op["H"]) + \
115                              H2(sys_enl_op["conn_Sz"], sys_enl_op["conn_Sp"], env_enl_op["conn_Sz"], env_enl_op["conn_Sp"])
```

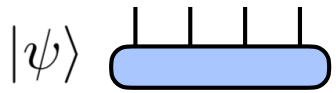
- form the Hamiltonian matrix for the superblock

system  environment

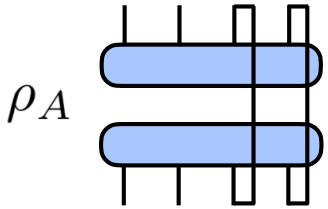
DMRG step (2)

```
117     # Call ARPACK to find the superblock ground state.  ("SA" means find the
118     # "smallest in amplitude" eigenvalue.)
119     (energy,), psi0 = eigsh(superblock_hamiltonian, k=1, which="SA")
120
121     # Construct the reduced density matrix of the system by tracing out the
122     # environment
123     #
124     # We want to make the (sys, env) indices correspond to (row, column) of a
125     # matrix, respectively.  Since the environment (column) index updates most
126     # quickly in our Kronecker product structure, psi0 is thus row-major ("C
127     # style").
128     psi0 = psi0.reshape([sys_enl.basis_size, -1], order="C")
129     rho = np.dot(psi0, psi0.conjugate().transpose())
130
131     # Diagonalize the reduced density matrix and sort the eigenvectors by
132     # eigenvalue.
133     evals, evecs = np.linalg.eigh(rho)
134     possible_eigenstates = []
135     for eval, evec in zip(evals, evecs.transpose()):
136         possible_eigenstates.append((eval, evec))
137     possible_eigenstates.sort(reverse=True, key=lambda x: x[0])  # largest eigenvalue first
```

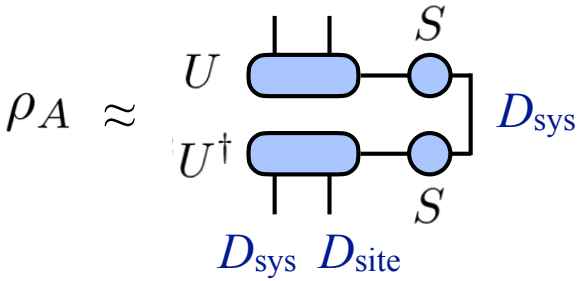
- diagonalize the superblock Hamiltonian to find the ground state



- form the reduced density matrix



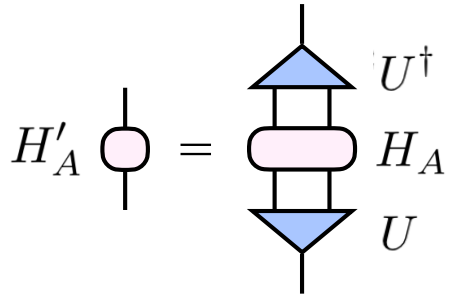
- diagonalize ρ_A to find U and S, and make a truncation to U and S

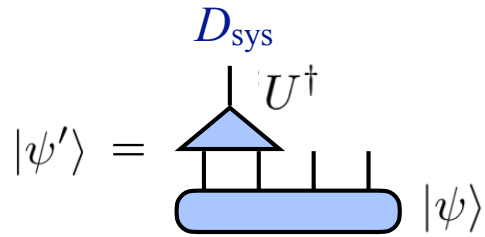


DMRG step (3)

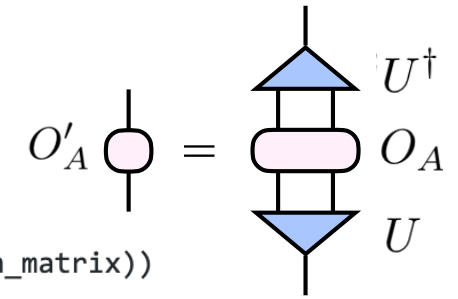
```
139        # Build the transformation matrix from the `m` overall most significant
140        # eigenvectors.
141        my_m = min(len(possible_eigenstates), m)
142        transformation_matrix = np.zeros((sys_enl.basis_size, my_m), dtype='d', order='F')
143        for i, (eval, evec) in enumerate(possible_eigenstates[:my_m]):
144            transformation_matrix[:, i] = evec
145
146        truncation_error = 1 - sum([x[0] for x in possible_eigenstates[:my_m]])
147        print("truncation error:", truncation_error)
148
149        # Rotate and truncate each operator.
150        new_operator_dict = {}
151        for name, op in sys_enl.operator_dict.items():
152            new_operator_dict[name] = rotate_and_truncate(op, transformation_matrix)
153
154        newblock = Block(length=sys_enl.length,
155                         basis_size=my_m,
156                         operator_dict=new_operator_dict)
157
158        return newblock, energy
```

- form the new system block using U

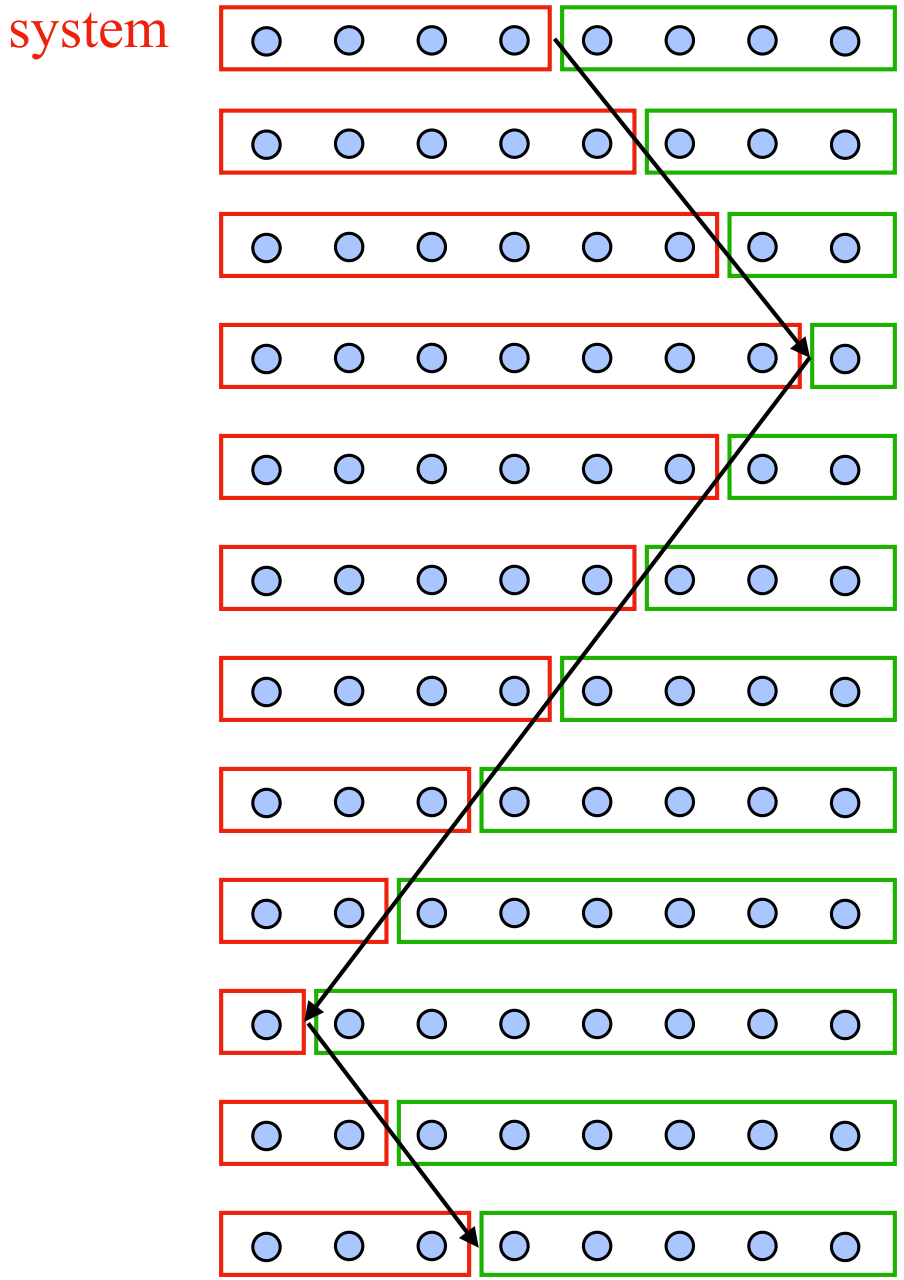





```
86  def rotate_and_truncate(operator, transformation_matrix):
87      """Transforms the operator to the new (possibly truncated) basis given by
88      `transformation_matrix`.
89      """
90      return transformation_matrix.conjugate().transpose().dot(operator.dot(transformation_matrix))
```



Traditional finite DMRG algorithm



- sweep back and forth until convergence is reached

Homework rules

- this homework has only one task
  please submit through http://learn.tsinghua.edu.cn

- please submit the source code and a detailed note including:
  (1) a brief summary of theory and algorithm
  (2) the structure and technical details of the code
  (3) problems encountered and solutions
  (4) summary of results and your understanding

- deadline is Nov. 3, 23:00
  if you submit after the deadline, you will have less points

- if you are an expert in traditional DMRG, this homework may be waived
  please contact me in person and submit something you have done before

Homework

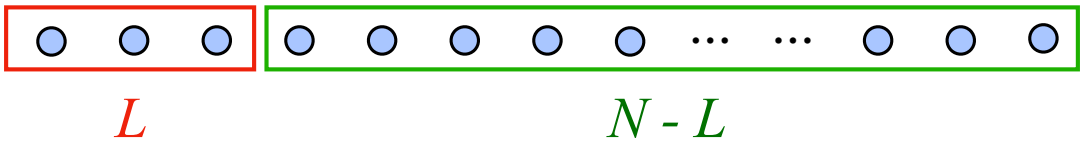
- consider the following spin model on a N-site chain

$$H = -\sum_{i=1}^{N-1} (\sigma_i^x \sigma_{i+1}^x + g\sigma_i^y \sigma_{i+1}^y)$$

please take the following parameters: $N = 40$,

(1) $m = 10$, $g \in [0.5,\ 1.0,\ 1.5]$, (2) $g = 1.0$, $m \in [10,\ 20,\ 30]$,

where m is the maximum number of states kept in the new basis



- based on the traditional finite size DMRG method, e.g., https://github.com/simple-dmrg/simple-dmrg (finite system algorithm)

(1) explicitly show the data of ground state energy and entanglement entropy $S(L)$

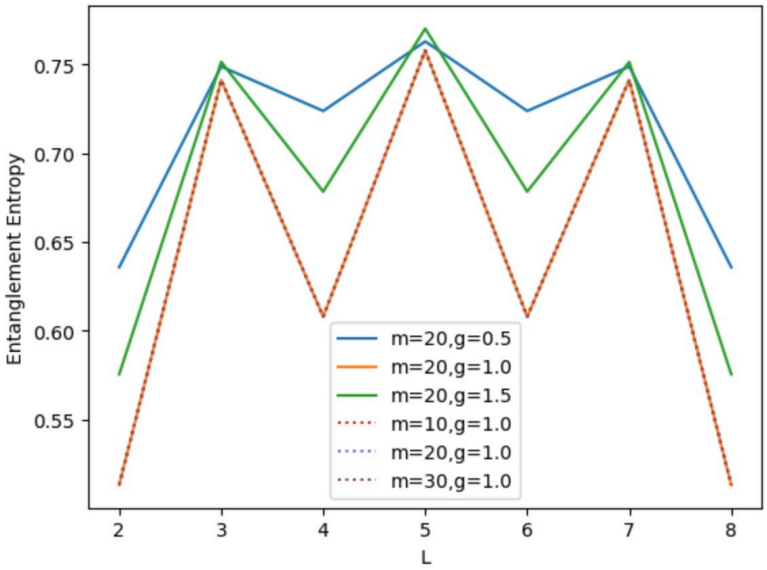
(2) plot the entanglement entropy $S(L)$ as a function of L, $S(L) = -\mathrm{Tr}[\rho_L \ln \rho_L]$

(3) for $m = 20$ and $g = 1.0$, fit $S(L)$ using $S(L) = \frac{c}{6} \ln(\frac{N}{\pi} \sin \frac{\pi L}{N}) + c'$ and find c

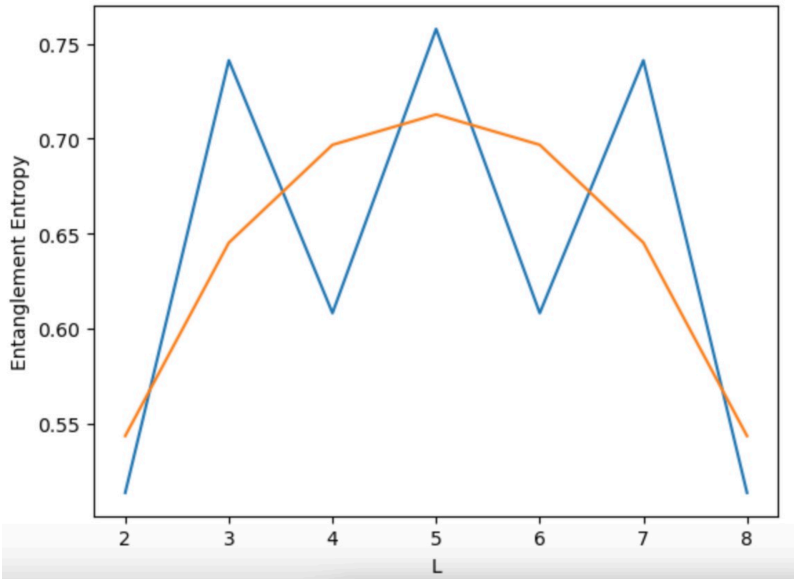
Sample output

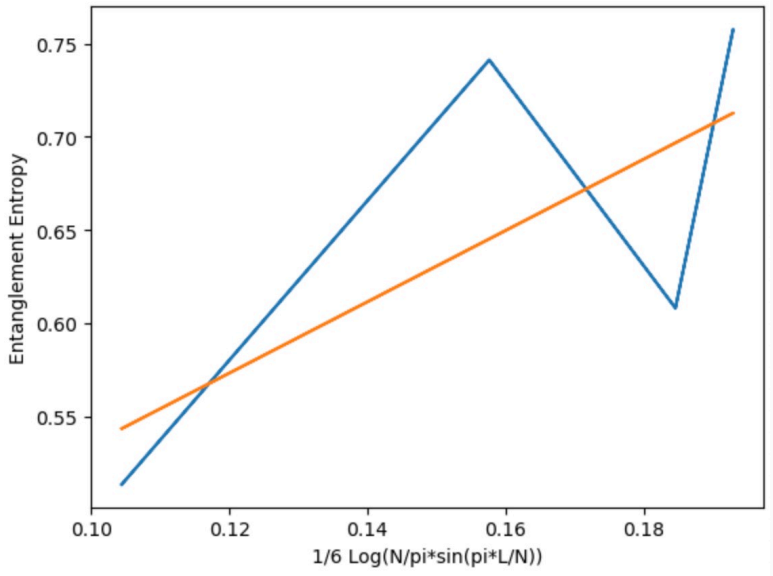
- please note the input parameters are different from homework

```
(1) and (2)
(N, m, g)= (10, 10, 0.5)
Energy: -9.72142811857606
L: [2 3 4 5 6 7 8]
EE: [0.635817 0.748776 0.723754 0.762894 0.723754 0.748776 0.635817]
(N, m, g)= (10, 10, 1.0)
Energy: -12.05331589444485
L: [2 3 4 5 6 7 8]
EE: [0.5133   0.74118  0.607996 0.757687 0.607996 0.74118  0.5133  ]
(N, m, g)= (10, 10, 1.5)
Energy: -15.487279359888992
L: [2 3 4 5 6 7 8]
EE: [0.575612 0.751367 0.678385 0.770133 0.678385 0.751367 0.575612]
(N, m, g)= (10, 10, 1)
Energy: -12.053315930956822
L: [2 3 4 5 6 7 8]
EE: [0.5133   0.74118  0.607996 0.757687 0.607996 0.74118  0.5133  ]
(N, m, g)= (10, 20, 1.0)
Energy: -12.053348692378805
L: [2 3 4 5 6 7 8]
EE: [0.513393 0.7413   0.608093 0.757841 0.608093 0.7413   0.513393]
(N, m, g)= (10, 30, 1.0)
Energy: -12.0533485804037
L: [2 3 4 5 6 7 8]
EE: [0.513393 0.7413   0.608093 0.757841 0.608093 0.7413   0.513393]
```




```
(3): central_charge= 1.913020938231267   intercept= 0.343606315741585   r_value= 0.6670297354242002
```





Grading of this homework

- academic integrity [+50]

  completing assignments independently, creating and expressing your own ideas, DON'T copy answers from others or allow others to copy your answers

- the source code can be executed [+10] and can provide correct results [+10]

  the source code has high readability [+10]

- there is a detailed note file to explain the basic principle, source code and results [+10]

  the note file is well-written and easy to understand [+10]

- did not calculate and fit entanglement entropy [-30]

- time-dependent score: after DDL: [-1 per day]

- self-motivation score: new ideas or extra results [+10]